Unit 09: Programming the Bourne Again Shell

CONTENTS

Objectives

Introduction

9.1 Control Structures

9.2 File Descriptor

9.3 Parameters and Variables

9.4 Builtin Commands

9.5 Expressions

9.6 Operators

9.7 Increment and Decrement

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Understand the control structures
- Understand the file descriptors
- Know the parameters and variables
- Understand the builtins
- Understand the expressions

Introduction

The programming languages are used for writing the programs. A simple program executes a sequence of statements without any jump or condition. In a programming language, we have various kinds of control structures which basically interrupts the flow of statements based upon conditions. The control flow commands alter the order of execution of commands within a shell script. It specifies the order in which computations are performed.

9.1 Control Structures

There are various control structures:

- if...then,
- for...in,
- while,
- until,
- break,

- Continue
- case statements

if...then

This control structure is used to express the decisions. The if...then control structure has the following syntax:

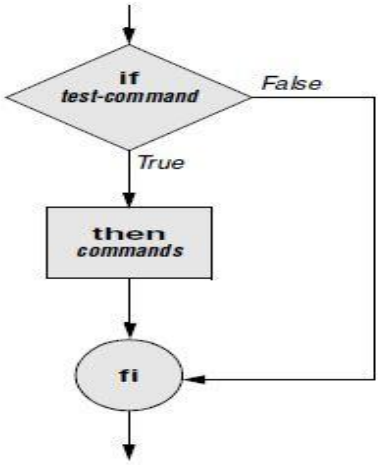
iftest-command

then

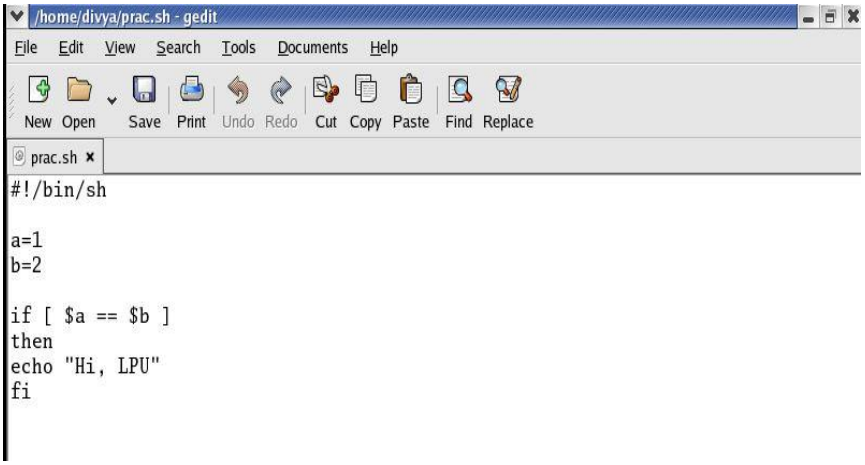
commands

fi

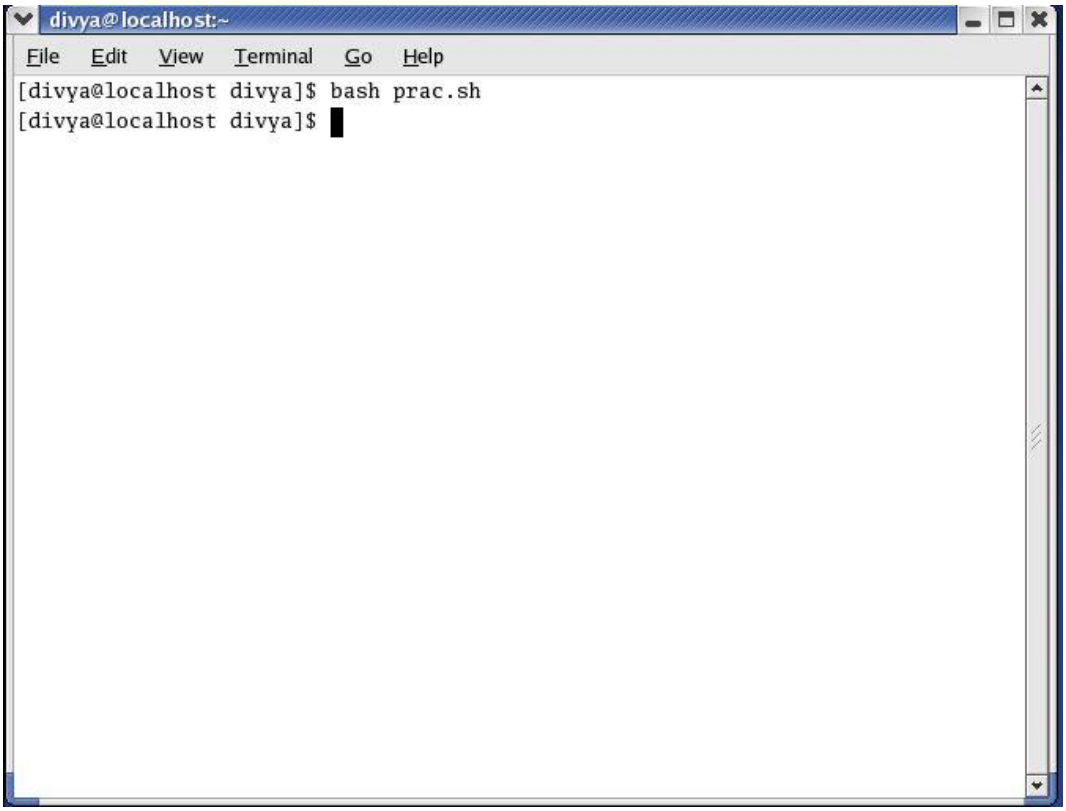
If the statement given in test_command turns out to be true, then the commands given must be executed. In this control structure, there is no command that needs to be printed when the test_command turns out to be false. The flow of if … then control structure is shown below:



Here, one example is taken in which the value of a is 1 and b is 2 and the test condition is to check whether both values are equal or not. If the condition is true, then only the statement "Hi, LPU" will be printed. Otherwise, the control will exit. The control structure is ended using fi.



The output of the program is " ". It will print nothing as the condition is false here. The control will just exit.



Task: Write a shell script to display square of a number

if...then...else

This control structure is also used to represent the decisions. Here the else part can be optional. The test-command will be evaluated. If this turns out to be true, then commands after that will be printed. If the test-commands turns out to be false, then the statements given in else will be printed. The **if...then...else control structure** has the following syntax:

if ***test-command***

then

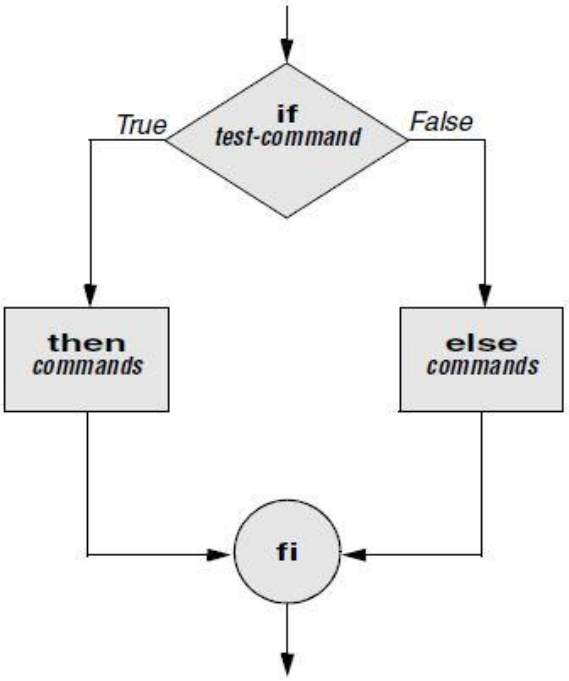
commands

else

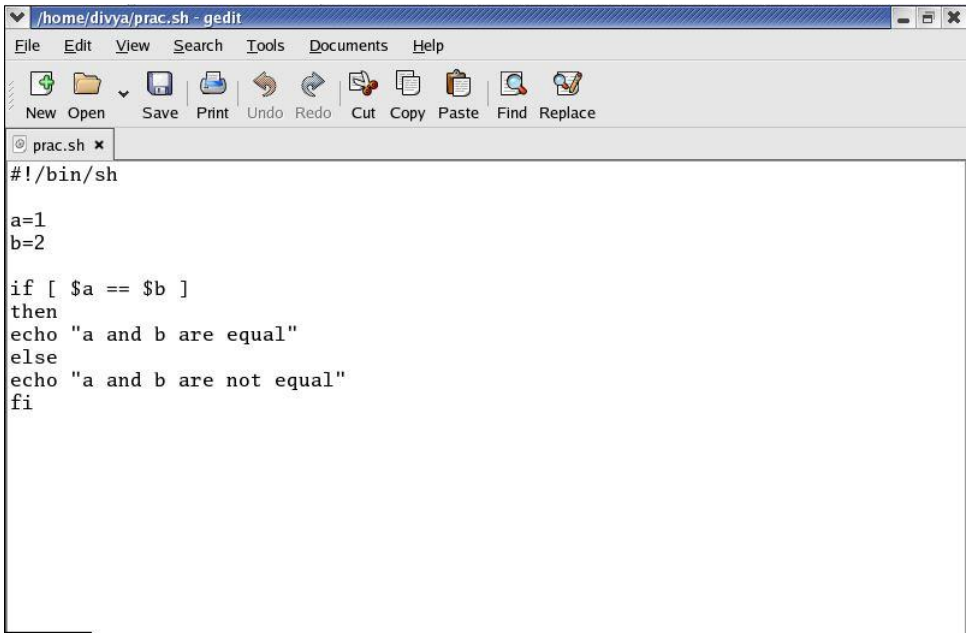
commands

fi

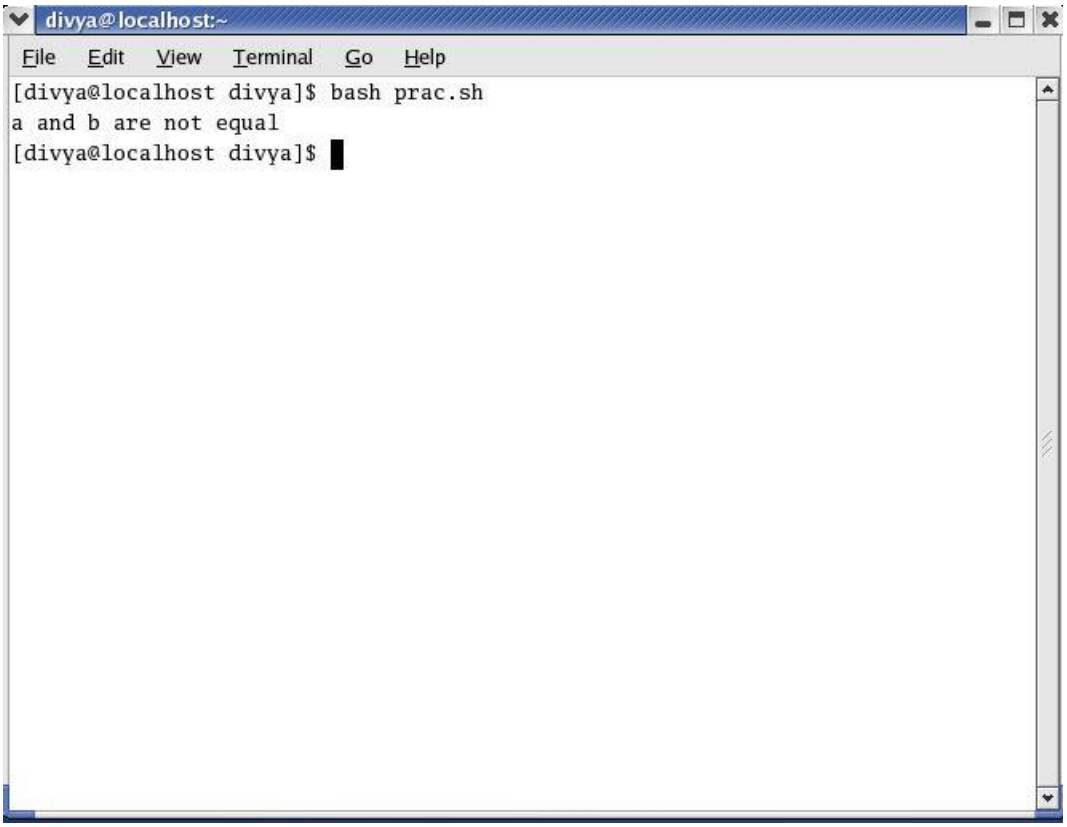
The flow of if… then… else is shown below:



Here in this example, two variables are taken, i.e., a and b. If the values of a and b are equal, then the statement "a and b are equal" otherwise "a and b are not equal" is printed. The structure is ended with fi.



The output of the program is: The value of a is 1 and b is 2. It will print "a and b are not equal".



Task: Write a shell script to check whether a person is eligible to vote or not.

if...then...elif

This sequence of if statements is the most general way of writing a multi-way decision. The expressions are evaluated in order; if an expression is true, the statement associated with it is executed, and this terminates the whole chain. As always, the code for each statement is either a single statement, or a group of them in braces. The last else part handles the ``none of the above'' or default case where none of the other conditions is satisfied.The **if...then...elif control structure** ***has the following*** syntax:

if ***test-command***

then

commands

elif ***test-command***

then

commands

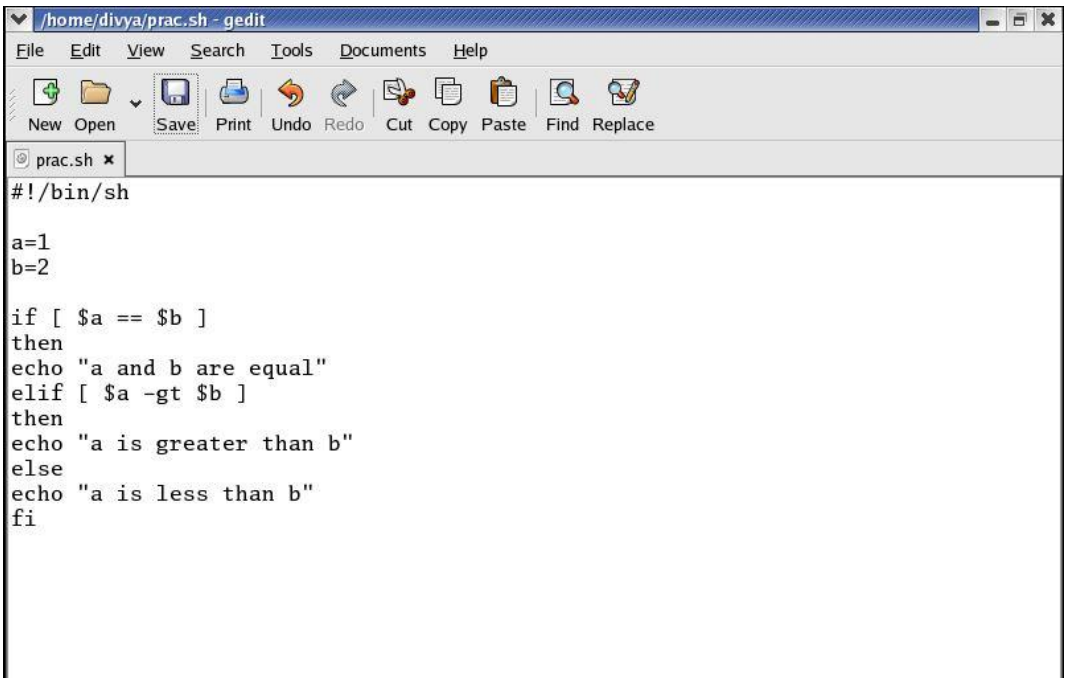
. . .

else

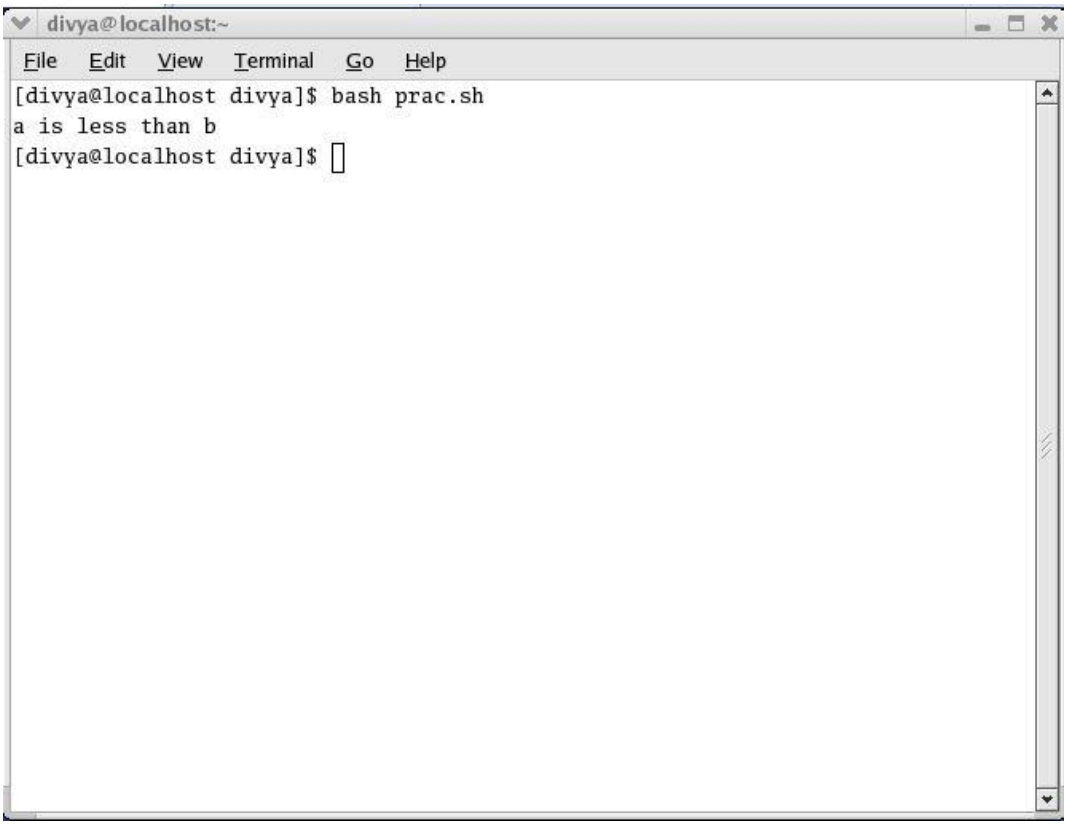
commands

fi

The values of a and b are taken. The condition is to check the value of and b. If both are equal, then it should print "a and b are equal", If not, it should check if a is greater than b, if yes, then it should print "a is greater than b". Otherwise "a is less than b".



The value of a is 1 and b is 2. So, the output of the program is "a is less than b".



Task: Write a shell script to display grades as per the following ranges of %age:

>= 80 'A+'

>=60 &&<80 'A'

> = 50 &&< 60 'B'

> = 40 &&<50 'C'

<40 'E'.

for...in

A loop is a sequence of instructions that is continually repeated until a certain condition is reached. It is a block of code that will repeat repeatedly. The for loop operates on lists of items. It repeats a set of commands for every item in a list. The for...in control structure *has the following syntax:*

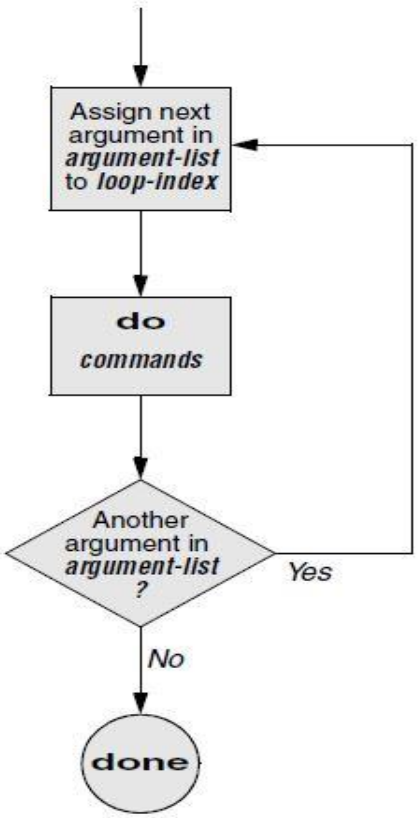
for ***loop-index in argument-list***

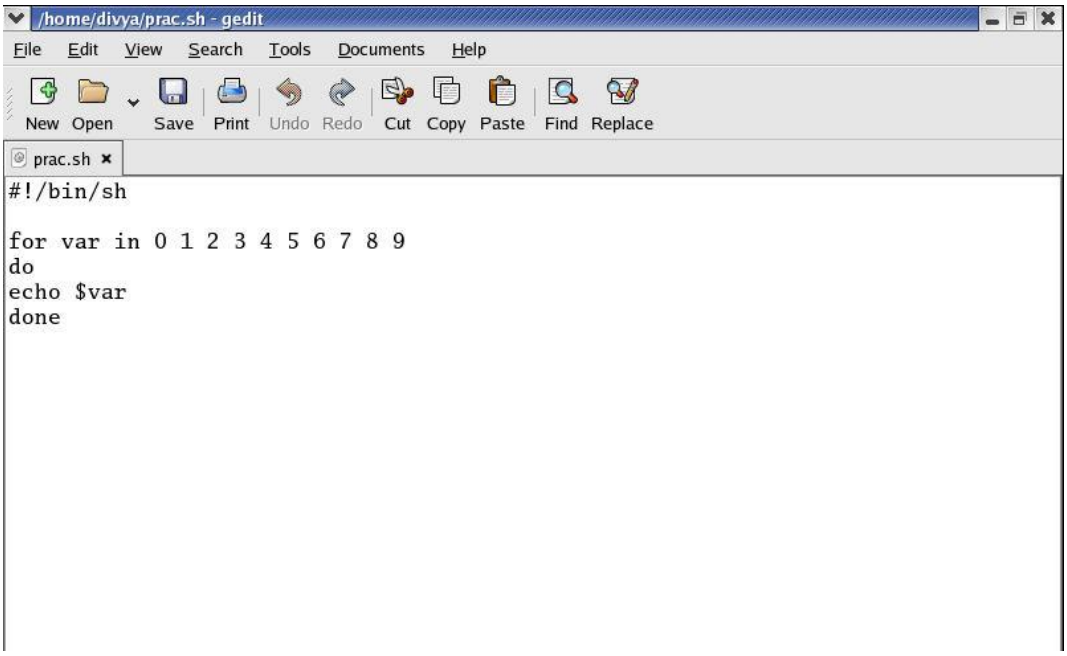
do

commands

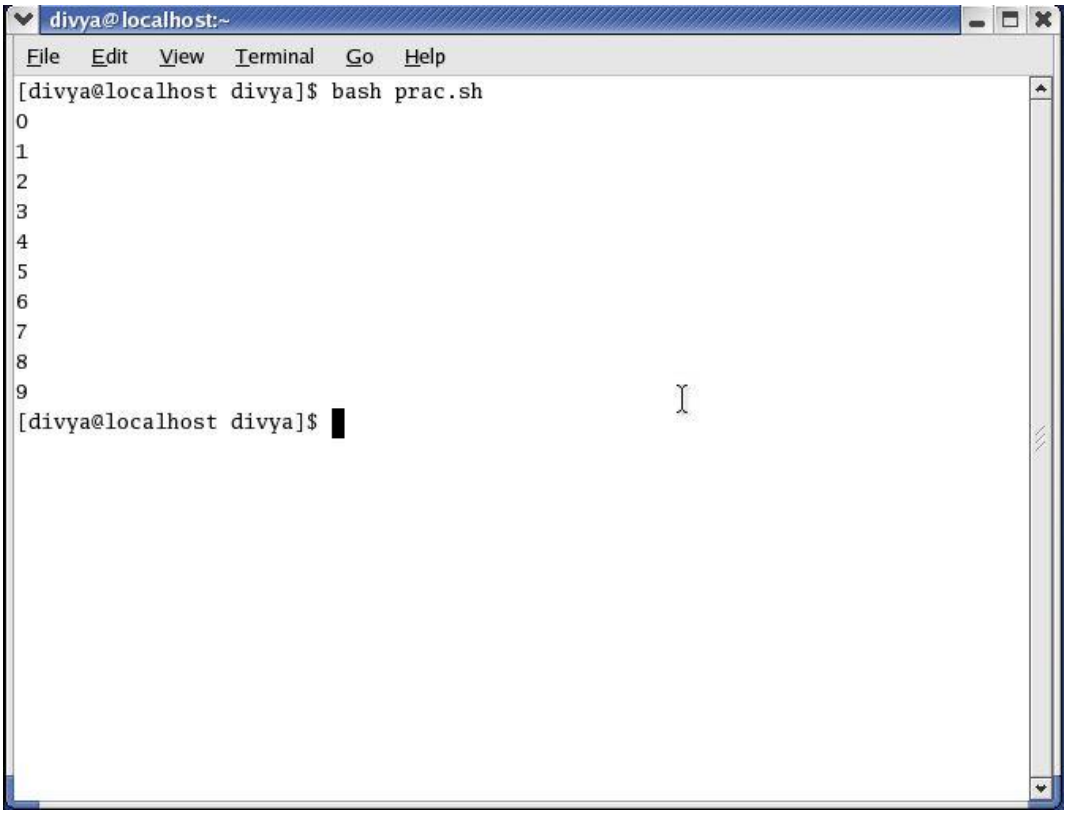
done

Here loop index is the variable you specify in the do section and will contain the item in the loop that you are on. The list of arguments can be anything that returns a space or newline separated list.





The output of the program is printing of numbers starting from 0 till 9.



Task: Write a shell script to sum of all numbers starting from 1 to 100 using for loop

While

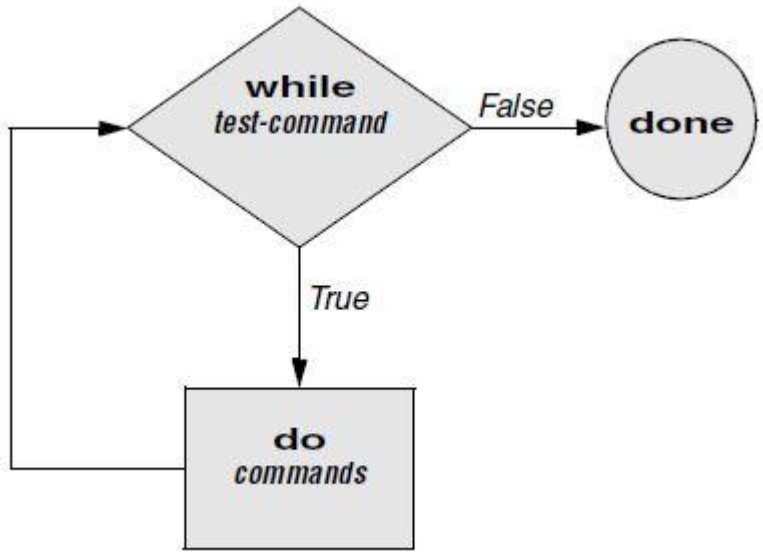
As long as the test-command (Figure 10-5) returns a true exit status, the while structure continues to execute the series of commands delimited by the do and done statements. Before each loop through the commands, the structure executes the test command. When the exit status of the test-command is false, the structure passes control to the statement after the done statement. The while control structure (not available in tcsh) has the following syntax:

while ***test-command***

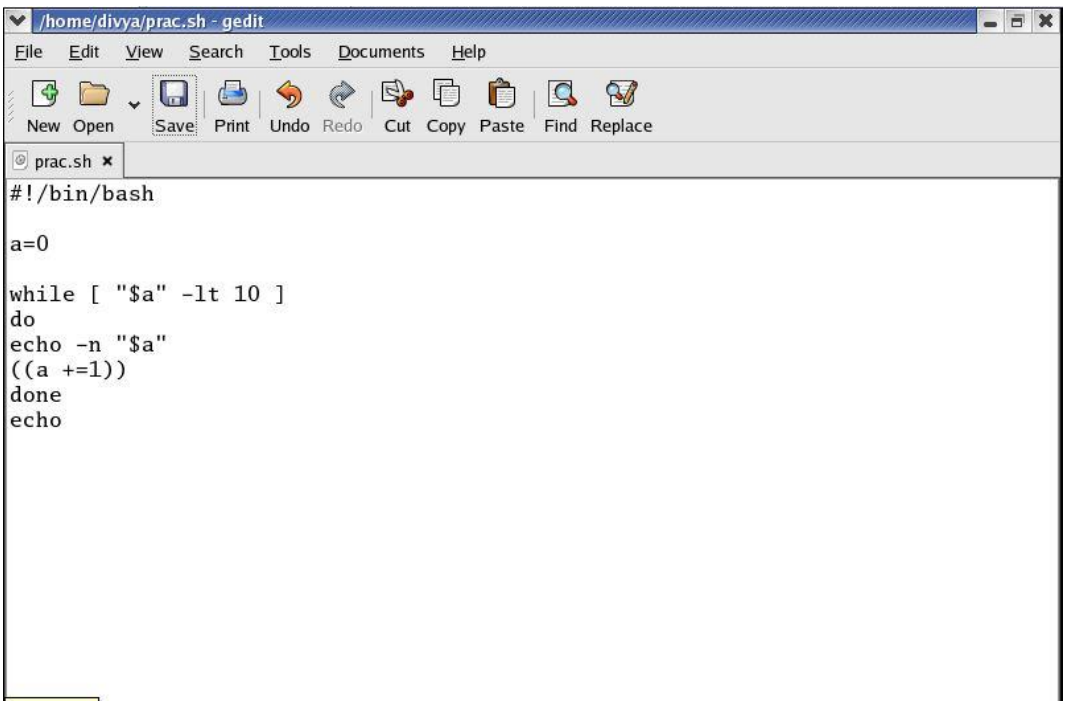
do

commands

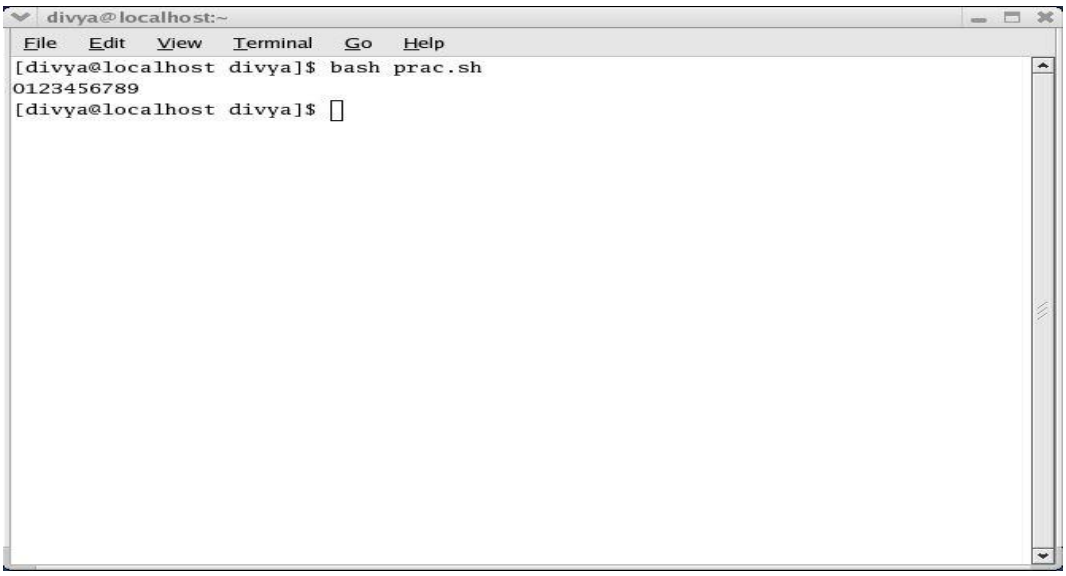
done



The variable a is taken and initialized with 0. Till the value of a is less than 10, it keeps on printing and incrementing the values.



The output of the program is: The values starting from 0 till 9 will be printed.



Task: Write a shell script to count odd numbers from 10 to 100.

Until

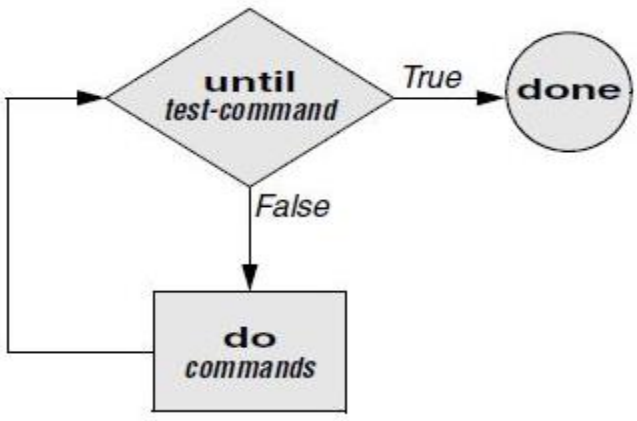
The until continues to loop until the test-command returns a true exit status. The while loop is perfect for a situation where you need to execute a set of commands while some condition is true. Sometimes you need to execute a set of commands until a condition is true. The syntax of until is:

until command

do

Statement(s) to be executed until command is true

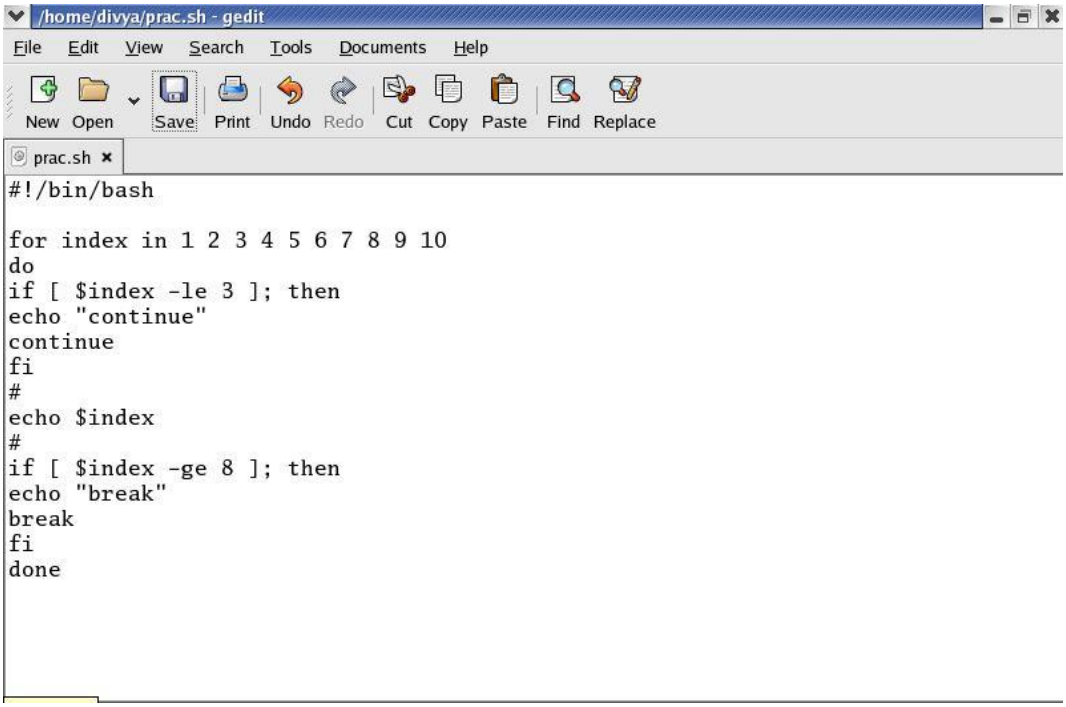
done

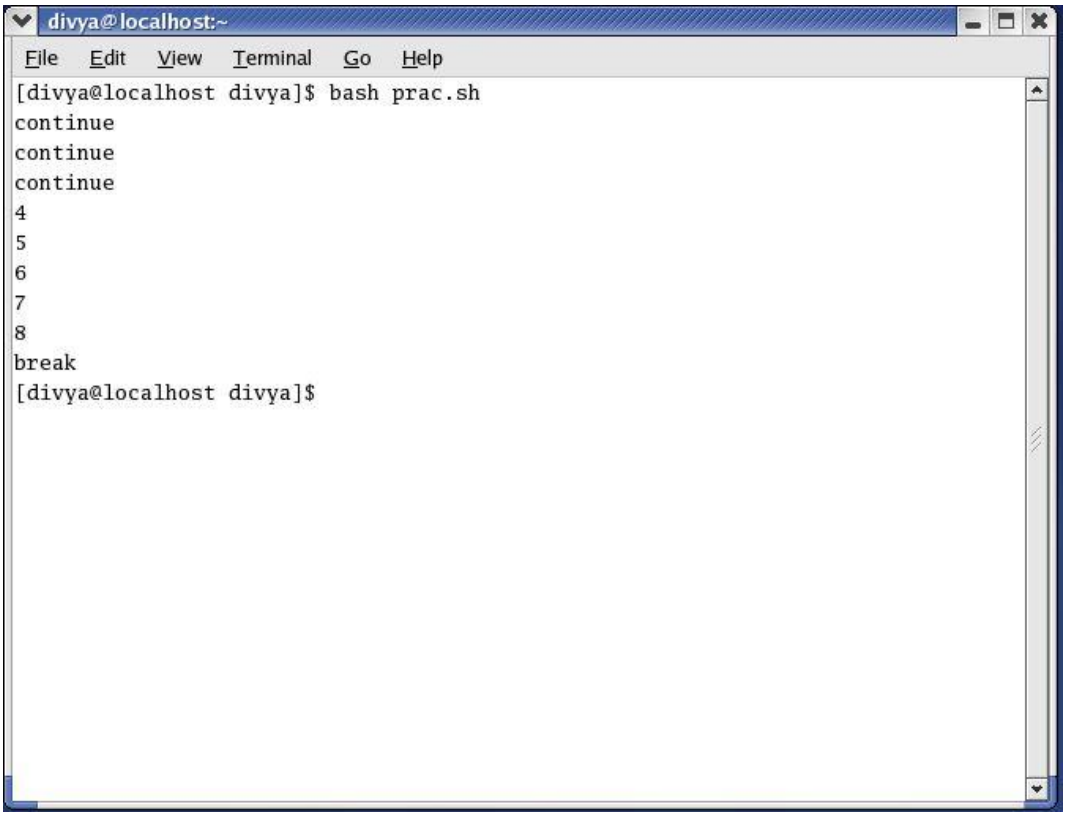


Task: Write a shell script to take input for a number and keep on counting the chances how many times user has not entered a valid number. 'Valid number is -99'.

break and continue

You can interrupt a for, while, or until loop by using a break or continue statement.The break statement transfers control to the statement after the done statement, thereby terminating execution of the loop. The continue command transfers control to the done statement, continuing execution of the loop.





Task: Write a shell script to display square of all numbers from m to n when first multiple of 10 is reached loop should terminate.

Case

You can use multiple **if...elif** statements to perform a multi way branch. However, this is not always the best solution, especially when all the branches depend on the value of a single variable. It supports **case...esac** statement which handles exactly this situation, and it does so more efficiently than repeated **if...elif** statements. The case structure is a multiple-branch decision mechanism. The path taken through the structure depends on a match or lack of a match between the test-string and one of the patterns. The pattern in the case structure is analogous to an ambiguous file reference. It can ne any special character like *, ?, […] or |.The case control structure has the following syntax:

case ***test-string in***

pattern-1)

commands-1

;;

pattern-2)

commands-2

;;

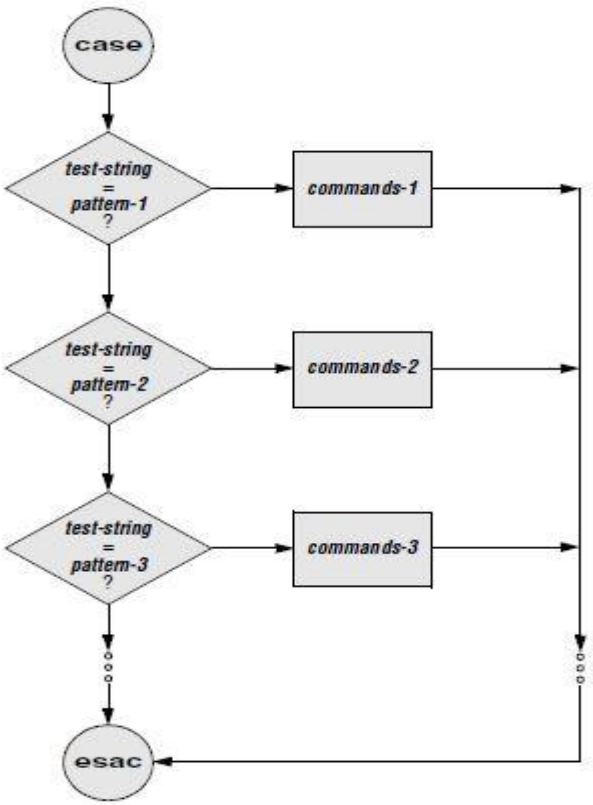
pattern-3)

commands-3

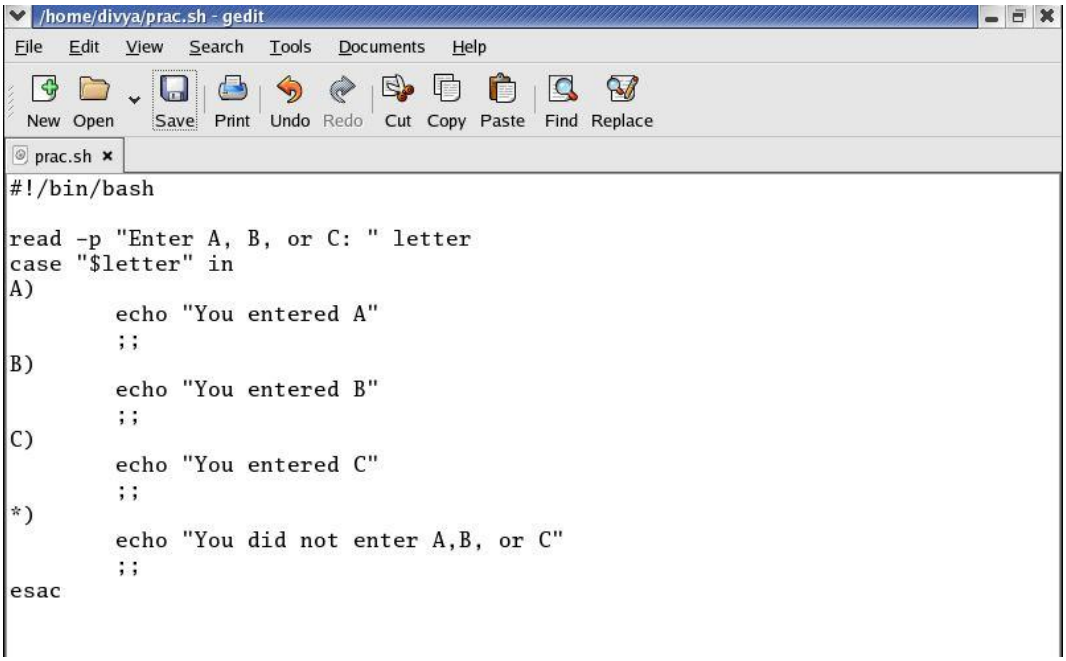
;;

. . .

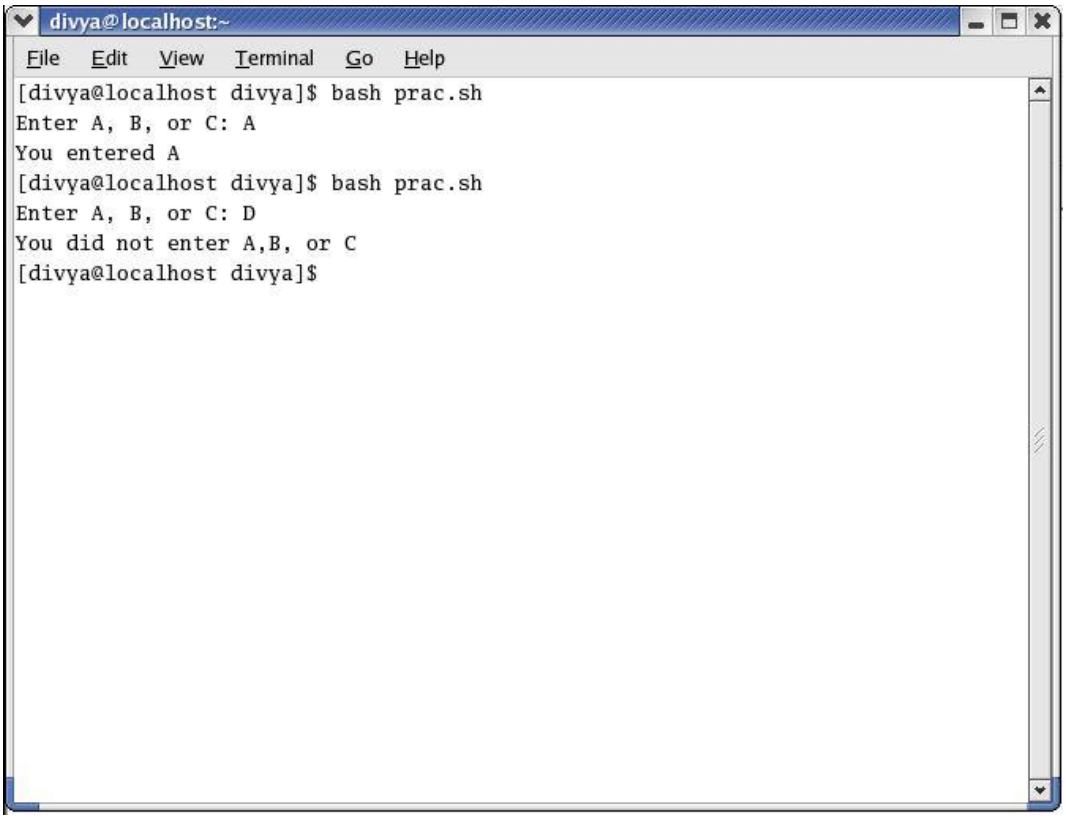
esac



The next program asks to enter any character: A,B or C. If you have entered A, it prints "You entered A". If you have entered B, it prints " You entered B". If you have entered C, it prints " You entered C". If any other character is entered, it prints " You did not enter A, B, or C". The structure ends with esac.



The output of the program is:



Task: Write a shell script to implement arithmetic calculator.

Select

The select control structure is based on the one found in the Korn Shell. It displays a menu, assigns a value to a variable based on the user's choice of items, and executes a series of commands. The select control structure has the following syntax:

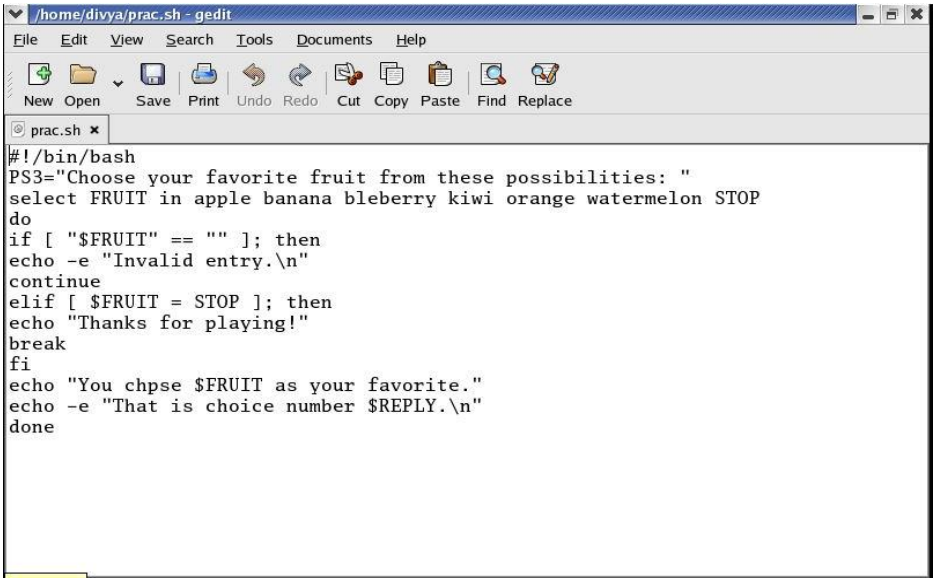
select ***varname [in arg . . .]***

do

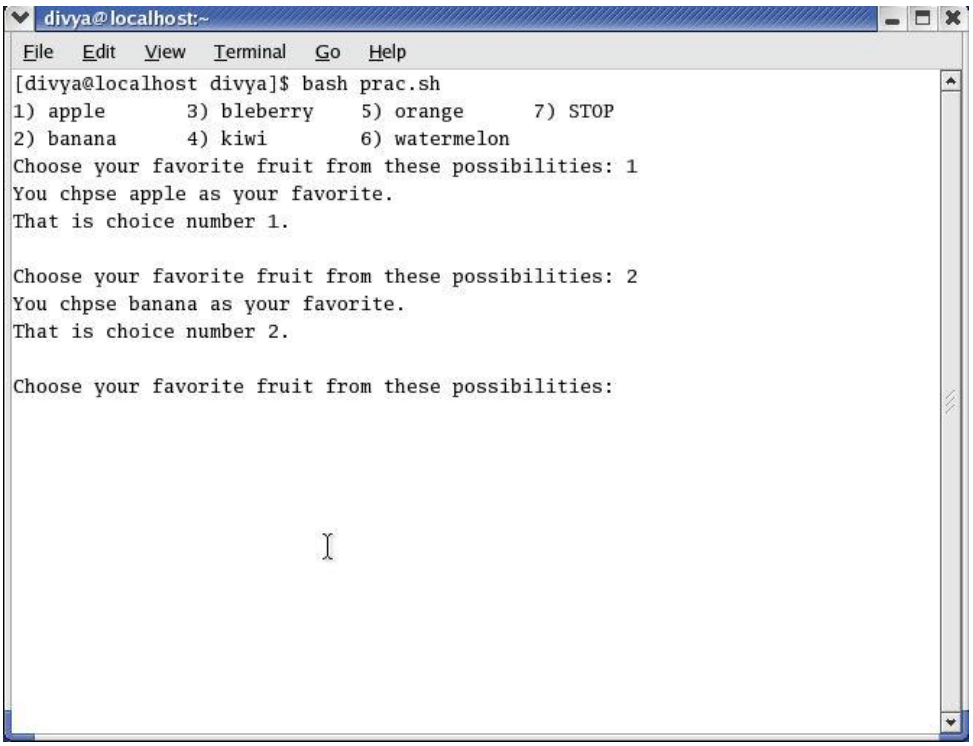
commands

done

The select structure displays a menu of the arg items. If you omit the keyword in and the list of arguments, select uses the positional parameters in place of the arg items. The menu is formatted with numbers before each item.



The output of the program is:



9.2 File Descriptor

Before a process can read from or write to a file, it must open that file. When a process opens a file, Linux associates a number (called a file descriptor) with the file. A file descriptor is an index into the process's table of open files. Each process has its own set of open files and its own file descriptors. After opening a file, a process reads from and writes to that file by referring to its file descriptor. When it no longer needs the file, the process closes the file, freeing the file descriptor. A typical Linux process starts with three open files: standard input (file descriptor 0), standard output (file descriptor 1), and standard error (file descriptor 2). Often these are the only files the process needs.

Opening a file descriptor

The Bourne Again Shell opens files using the exec built in as follows: exec n> outfileopens outfile for output and holds it open, associating it with file descriptor n. The exec m< infileopens infile for input and holds it open, associating it with file descriptor m.

Duplicating a file descriptor

The <& token duplicates an input file descriptor; >& duplicates an output file descriptor. You can duplicate a file descriptor by making it refer to the same file as another open file descriptor, such as standard input or output. To open or redirect file descriptor n as a duplicate of file descriptor m: exec n<&m

Once you have opened a file, you can use it for input and output in two ways. First, you can use I/O redirection on any command line, redirecting standard output to a file descriptor with >&n or redirecting standard input from a file descriptor with <&n. Second, you can use the read and echo builtins. If you invoke other commands, including functions, they inherit these open files and file descriptors. When you have finished using a file, you can close it using: exec n<&–

9.3 Parameters and Variables

There are various parameters and variables which are used. These are:

- Array Variables
- Locality of Variables
- Functions
- Special Parameters
- Positional Parameters

Array Variables

The Bourne Again Shell supports one-dimensional array variables. The subscripts are integers with zero-based indexing (i.e., the first element of the array has the subscript 0). The declaration and assignment of values to an array can be done as:name=(element1 element2 ...). An example of assigning four values to the array NAMES can be done as:$ NAMES=(max helen sam zach). It references a single element of an array as follows:$ echo ${NAMES[2]}

sam

The declare builtin with the –a option displays the values of the arrays (and reminds you that bash uses zero based indexing for arrays):

```
$ A=("${NAMES[*]}")
$ B=("${NAMES[@]}")
$ declare -a
declare -a A='([0]="max helen sam zach")'
declare -a B='([0]="max" [1]="helen" [2]="sam" [3]="zach")'
```

...

declare -a NAMES='([0]="max" [1]="helen" [2]="sam" [3]="zach")'

Locality of Variables

By default, variables are local to the process in which they are declared. Thus, a shell script does not have access to variables declared in your login shell unless you explicitly make the variables available (global). Under bash, export makes a variable available to child processes. Under tcsh, setenv assigns a value to a variable and makes it available to child processes.

Locality of Variables(Without export)

$ **cat extest1**

cheese=american

echo "extest1 1: $cheese"

subtest

echo "extest1 2: $cheese"

$ **cat subtest**

echo "subtest 1: $cheese"

cheese=swiss

echo "subtest 2: $cheese"

$ **./extest1**

extest1 1: american

subtest 1:

subtest 2: swiss

extest1 2: American

Locality of Variables(With export)

$ **cat extest2**

export cheese=american

echo "extest2 1: $cheese"

subtest

echo "extest2 2: $cheese"

$ **./extest2**

extest2 1: american

subtest 1: american

subtest 2: swiss

extest2 2: American

An export builtin can optionally include an assignment:export cheese=American. The preceding statement is equivalent to the following two statements:

cheese=american

export cheese

An export builtin can optionally include an assignment:export cheese=american

The preceding statement is equivalent to the following two statements:

```
cheese=american
export cheese
```

Functions

Because functions run in the same environment as the shell that calls them, variables are implicitly shared by a shell and a function it calls.

```
$ function nam () {
>echo $myname
>myname=zach
>}
$ myname=sam
$ nam
sam
$ echo $myname
zach
```

The myname variable is set to sam in the interactive shell. The nam function then displays the value of myname (sam) and sets myname to zach. The final echo shows that, in the interactive shell, the value of myname has been changed to zach. Local variables are helpful in a function written for general use. Because the function is called by many scripts that may be written by different programmers, you need to make sure the names of the variables used within the function do not conflict with (i.e., duplicate) the names of the variables in the programs that call the function. Local variables eliminate this problem. When used within a function, the typeset builtin declares a variable to be local to the function it is defined in.

Special parameters

Special parameters enable you to access useful values pertaining to command-line arguments and the execution of shell commands. You reference a shell special parameter by preceding a special character with a dollar sign ($).As with positional parameters, it is not possible to modify the value of a special parameter by assignment.

$$: PID Number

The shell stores in the $$ parameter the PID number of the process that is executing it. In this example, echo displays the value of this variable and the ps utility confirms its value. Both commands show that the shell has a PID number of 5209:

```
$ echo $$
5209
$ ps
PID TTY TIME CMD
5209 pts/1 00:00:00 bash
6015 pts/1 00:00:00 ps
```

Because echo is built into the shell, the shell does not create another process when you give an echo command. However, the results are the same whether echo is a built in or not, because the shell

substitutes the value of $$ before it forks a new process to run a command. Incorporating a PID number in a filename is useful for creating unique file names when the meanings of the names do not matter; this technique is often used in shell scripts for creating names of temporary files. When two people are running the same shell script, having unique filenames keeps the users from inadvertently sharing the same temporary file.

$?: Exit Status

When a process stops executing for any reason, it returns an exit status to its parent process. The exit status is also referred to as a condition code or a return code. The $? ($status under tcsh) variable stores the exit status of the last command. By convention a nonzero exit status represents a false value and means the command failed. A zero is true and indicates the command executed successfully. The first ls command succeeds and the second fails, as demonstrated by the exit status:

$ ls es

es

$ echo $?

0

$ ls xxx

ls: xxx: No such file or directory

$ echo $?

1

You can specify the exit status that a shell script returns by using the exit builtin, followed by a number, to terminate the script. If you do not use exit with a number to terminatea script, the exit status of the script is that of the last command the script ran.

$ cat es
echo This program returns an exit status of 7.
exit 7
$ es
This program returns an exit status of 7.
$ echo $?
7
$ echo $?

0

Positional Parameters

Positional parameters comprise the command name and command-line arguments. These parameters are called positional because within a shell script, you refer to them by their position on the command line. Only the **set** builtin allows you to change the values of positional parameters. However, you cannot change the value of the command name from within a script. The tcsh set builtin does not change the values of positional parameters.

$#: Number of Command-Line Arguments

The $# parameter holds the number of arguments on the command line (positional parameters), not counting the command itself:

$ cat num_args

echo "This script was called with $# arguments."

$ ./num_args sam max zach

This script was called with 3 arguments.

$0: Name of the Calling Program

The shell stores the name of the command you used to call a program in parameter $0. This parameter is numbered zero because it appears before the first argument on the command line:

$ **cat abc**

echo "The command used to run this script is $0"

$ **./abc**

The command used to run this script is ./abc

$ **~sam/abc**

The command used to run this script is /home/sam/abc

$1–$n: Command-Line Arguments

The first argument on the command line is represented by parameter $1, the second argument by $2, and so on up to $n. For values of n greater than 9, the number must be enclosed within braces. For example, the twelfth command-line argument is represented by ${12}. The following script displays positional parameters that hold command-line arguments:

$ cat display_5args

echo First 5 arguments are $1 $2 $3 $4 $5

$ ./display_5args zach max helen

First 5 arguments are zach max helen

shift: Promotes Command-Line Arguments

The shift builtin promotes each command-line argument. The first argument (which was $1) is discarded. The second argument (which was $2) becomes the first argument (now $1), the third becomes the second, and so on. Because no "unshift" command exists, you cannot bring back arguments that have been discarded. An optional argument to shift specifies the number of positions to shift (and the number of arguments to discard); the default is 1.

The following demo_shift script is called with three arguments. Double quotation marks around the arguments to echo preserve the spacing of the output. The program displays the arguments and shifts them repeatedly until no more arguments are left to shift:

$ **cat demo_shift**

echo "arg1= $1 arg2= $2 arg3= $3"

shift

echo "arg1= $1 arg2= $2 arg3= $3"

shift

echo "arg1= $1 arg2= $2 arg3= $3"

shift

echo "arg1= $1 arg2= $2 arg3= $3"

shift

$ **./demo_shift alice helen zach**

arg1= alice arg2= helen arg3= zach

arg1= helen arg2= zach arg3=

arg1= zach arg2= arg3=

arg1= arg2= arg3=

set: Initializes Command-Line Arguments

When you call the set builtin with one or more arguments, it assigns the values of the arguments to the positional parameters, starting with $1 (not available in tcsh). The following script uses set to assign values to the positional parameters $1, $2, and $3:

$ cat set_it

set this is it

echo $3 $2 $1

$ ./set_it

it is this

$* and $@: Represent All Command-Line Arguments

The $* parameter represents all command-line arguments, as the display_all program demonstrates:

$ cat display_all

echo All arguments are $*

$ ./display_all a b c d e f g h i j k l m n o p

All arguments are a b c d e f g h i j k l m n o p

9.4 Builtin Commands

Builtin commandsdo not fork a new processwhen you execute them.

Builtins	Funcexittions
:	Returns 0 or *true*
.	Executes a shell script as part of the current process
bg	Puts a suspended job in the background
break	Exits from a looping control structure
cd	Changes to another working directory
continue	Starts with the next iteration of a looping control structure

echo	Displays its arguments
eval	Scans and evaluates the command line
exec	Executes a shell script or program in place of the current process
export	Exits from the current shell
fg	Brings a job from the background into the foreground
getopts	Parses arguments to a shell script
jobs	Displays a list of background jobs
kill	Sends a signal to a process or job
pwd	Displays the name of the working directory
read	Reads a line from standard input
readonly	Declares a variable to be readonly
set	Sets shell flags or command-line argument variables; with no argument, lists all variables
shift	Promotes each command-line argument
test	Compares arguments
times	Displays total times for the current shell and its children
trap	Traps a signal
type	Displays how each argument would be interpreted as a command

umask	Returns the value of the file-creation mask
unset	Removes a variable or function
wait	Waits for a background process to terminate

9.5 Expressions

An expression comprises constants, variables, and operators that the shell can process to return a value. It contains arithmetic, logical and conditional expressions and operators.

Arithmetic Evaluation

The Bourne Again Shell can perform arithmetic assignments and evaluate many different types of arithmetic expressions, all using integers. The shell performs arithmetic assignments in a number of ways.

One is with arguments to the let builtin:$ **let "VALUE=VALUE * 10 + NEW".** Within a let statement you do not need to use dollar signs ($) in front of variable names. Double quotation marks must enclose a single argument, or expression, that contains SPACEs. Because most expressions contain SPACEs and need to be quoted, bash accepts ((expression)) as a synonym for let "expression", obviating the need for both quotation marks and dollar signs: $ ((VALUE=VALUE * 10 + NEW))

Logical expressions

You can use the ((expression)) syntax for logical expressions, although that task is frequently left to [[expression]].

$ **cat age2**

#!/bin/bash

echo -n "How old are you? "

read age

if ((30 < age && age < 60)); then

echo "Wow, in $((60-age)) years, you'll be 60!"

else

echo "You are too young or too old to play."

fi

$ **./age2**

How old are you? **25**

You are too young or too old to play.

Logical Evaluation (Conditional expressions)

The syntax of a conditional expression is [[expression]]where expression is a Boolean (logical) expression. You must precede a variable name with a dollar sign ($) within expression. The result of executing this builtin, as with the test builtin, is a return status. The conditions allowed within the brackets are almost a superset of those accepted by test. Where the test builtin uses –a as a

Boolean AND operator, [[expression]] uses &&. Similarly, where test uses –o as a Boolean OR operator, [[expression]] uses ||.

String comparisons

The test builtin tests whether strings are equal. The [[expression]] syntax adds comparison tests for string operators. The > and < operators compare strings for order (for example, "aa" < "bbb"). The = operator tests for pattern match, not just equality: [[string = pattern]] is true if string matches pattern. This operator is not symmetrical; the pattern must appear on the right side of the equal sign. For example,

[[artist = a*]] is true (= 0), whereas [[a* = artist]] is false (= 1):

```
$ [[ artist = a* ]]
$ echo $?
0
$ [[ a* = artist ]]
$ echo $?
1
```

String Pattern Matching

The Bourne Again Shell provides string pattern-matching operators that can manipulate pathnames and other strings. These operators can delete from strings prefixes or suffixes that match patterns.

Operator	Function
#	Removes minimal matching prefixes
##	Removes maximal matching prefixes
%	Removes minimal matching suffixes
%%	Removes maximal matching suffixes

The syntax for these operators is: **${varname op pattern}.**

where op is one of the operators and pattern is a match pattern like that used for filename generation.

9.6 Operators

Arithmetic expansion and arithmetic evaluation in bash use the same syntax, precedence, and associativity of expressions as in the C language. Within an expression you can use parentheses to change the order of evaluation.

Type of Operator	Function
Post	
	Var++ Postincrement Var-- Postdecrement
Pre	
	++var Preincrement --var Predecrement
Unary	
	+ Unary Plus - Unary Minus
Negation	
	! Boolean NOT ~ Complement
Exponentiation	
	** Exponent
Multiplication, Division, Remainder	
	* Multiplication
	/ Division
	% Remainder
Addition, Subtraction	
	+ Addition
	- Subtraction

Bitwise Shifts	
	<< Left bitwise shift
	>> Right bitwise shift
Comparison	
	<= Less than or equal to
	>= Greater than or equal to
	< less than
	> Greater than
Equality, inequality	
	== Equality
	!= Inequality
Bitwise	
	* Bitwise AND
	^ Bitwise XOR
	| Bitwise OR
Boolean (Logical)	
	&& Boolean AND
	|| Boolean OR
Conditional Evaluation	
	?: Ternary Operator
Assignment	

	=, *=, /=, %=, +=, -=, <<=, >>=, &=, ^=, |= Assignment
Comma	
	, Comma

Pipe

The pipe token has higher precedence than operators. You can use pipes anywhere in a command that you can use simple commands. For example, the command line:

$ **cmd1 | cmd2 || cmd3 | cmd4 && cmd5 | cmd6**

is interpreted as if you had typed

$ **((cmd1 | cmd2) || (cmd3 | cmd4)) && (cmd5 | cmd6)**

9.7 Increment and Decrement

The post increment, post decrement, pre increment, and pre decrement operators work with variables. The pre- operators, which appear in front of the variable name(as in ++COUNT and –– VALUE), first change the value of the variable (++ adds 1;–– subtracts 1) and then provide the result for use in the expression. The post- operators appear after the variable name (as in COUNT++ and VALUE––); they first provide the unchanged value of the variable for use in the expression and then change the value of the variable.

Remainder

The remainder operator (%) yields the remainder when its first operand is divided by its second.

Boolean

The result of a Boolean operation is either 0 (false) or 1 (true). The && (AND) and || (OR) Boolean operators are called short-circuiting operators. If the result of using one of these operators can be decided by looking only at the left operand, the right operand is not evaluated. The && operator causes the shell to test the exit status of the command preceding it. If the command succeeded, bash executes the next command; otherwise, it skips the remaining commands on the command line. You can use this construct to execute commands conditionally.

Ternary

The ternary operator, ? :, decides which of two expressions should be evaluated, based on the value returned by a third expression:**expression1 ? expression2 : expression3**

Assignment: The assignment operators, such as +=, are shorthand notations. For example, N+=3 is the same as ((N=N+3)).

Summary

- The while loop is perfect for a situation where you need to execute a set of commands while some condition is true.
- You can interrupt a for, while, or until loop by using a break or continue statement.

- The break statement transfers control to the statement after the done statement, thereby terminating execution of the loop.
- The continue command transfers control to the done statement, continuing execution of the loop.
- A file descriptor is an index into the process's table of open files.
- A typical Linux process starts with three open files: standard input (file descriptor 0), standard output (file descriptor 1), and standard error (file descriptor 2).
- The <& token duplicates an input file descriptor; >& duplicates an output file descriptor.
- The Bourne Again Shell supports one-dimensional array variables.
- By default, variables are local to the process in which they are declared.
- Special parameters enable you to access useful values pertaining to command-line arguments and the execution of shell commands.
- Positional parameters comprise the command name and command-line arguments. These parameters are called positional because within a shell script, you refer to them by their position on the command line.
- An expression comprises constants, variables, and operators that the shell can process to return a value.

Keywords

- **Break:** The break statement transfers control to the statement after the done statement, thereby terminating execution of the loop.
- **Continue:** The continue command transfers control to the done statement, continuing execution of the loop.
- **File Descriptor**: A file descriptor is an index into the process's table of open files.
- **exec n> outfile:** It opens outfile for output and holds it open, associating it with file descriptor n.
- **exec m< infile:** It opens infile for input and holds it open, associating it with file descriptor m.
- **exec n<&–**: When you have finished using a file, you can close it using:exec n<&–
- **export**: Under bash, export makes a variable available to child processes.
- **Setenv**: Under tcsh, setenv assigns a value to a variable and makes it available to child processes.
- **Pipe**:The pipe token has higher precedence than operators. You can use pipes anywhere in a command that you can use simple commands.

Self Assessment

1. Continue statement

A. Breaks loop and goes to next statement after loop

B. does not break loop but starts new iteration

C. exits the program

D. Starts from beginning of program

2. >& duplicates

A. Input file descriptor

B. Output file descriptor

C. Error file descriptor

D. None of the above

3. << represents

A. Left bitwise shift

B. Right bitwise shift

C. Centre bitwise shift

D. None of the above

4. Which of these is assignment operator?

A. =

B. *=

C. /=

D. All of the above

5. Which of these builtin removes a variable or function?

A. set

B. unset

C. mask

D. umask

6. A typical Linux process has

A. File descriptor 0

B. File descriptor 1

C. File descriptor 2

D. All of the above mentioned

7. Before a file can read/write to a file, it must _____ the file.

A. Open

B. Close

C. Check

D. None of the above

8. We can reference a shell special parameter by preceding a special character with a _____

A. !

B. @

C. #

D. $

9. Which of these operators has the higher precedence?

A. Pipe

B. AND

C. OR

D. NOT

10. Instead of using if…..else multiple times, we can use one _____.

A. case….esac

B. if….fi

C. else……else

D. None of the above

11. Which of these control structures are available in Linux?

A. If…….then

B. For…….in

C. While

D. All of the above mentioned

12. The file descriptor is associated with _____

A. Opening of file

B. Reading from file

C. Writing from file

D. None of the above mentioned

13. The loops can be interrupted by using

A. break

B. continue

C. Both break and continue

D. None of the above

14. ^ represents

A. Bitwise AND

B. Bitwise OR

C. Bitwise XOR

D. None of the above

15. Which of these builtin removes a variable or function?

A. set

B. unset

C. mask

D. umask

Answers for Self Assessment

1.	B	2.	B	3.	A	4.	B	5.	B
6.	D	7.	D	8.	A	9.	A	10.	A
11.	D	12.	A	13.	C	14.	C	15.	B

Review Questions

1. Write a shell script to use a switch statement to process a menu selection, using both upper- and lower-case options.
2. Write a shell script that displays the names of all directory files, but no other types of files, in the working directory.
3. What is a control structure? Explain different types of control structures with examples.
4. Explain the syntax of while and for…in loop with examples.
5. What is a file descriptor? How can we open and duplicate a file descriptor?
6. What are special and positional parameters in Linux?
7. What is a builtin? Give ten examples of builtin commands.

Further Readings

Mark G. Sobell, A Practical Guide to Linux Commands, Editors and Shell Scripting, Second Edition, Prentice Hall.

Web Links

https://eng.libretexts.org/Bookshelves/Computer_Science/Operating_Systems/Linux_-_The_Penguin_Marches_On_(McClanahan)/13%3A_Working_with_Bash_Scripts/4.10%3A_Shell_Control_Statements